

Complete Flow: Doctor/Staff Signup & Approval System

Complete End-to-End Flow

DOCTOR/STAFF SIGNUP & APPROVAL FLOW	
(Step-by-Step Journey)	
PHASE 1: USER SIGNUP (Public - No Authentication Required)	
Step 1: User Opens Signup Page	
URL: https://yourapp.com/signup	
User sees form with:	
SELECT ROLE:  DOCTOR Medical Practitioner  STAFF Administrative Staff	
COMMON FIELDS (Required for both):	
• Full Name: []	
• Email: []	
• Password: []	
• Phone: [] (optional)	
DOCTOR-ONLY FIELDS (if Doctor selected):	
• Registration #: [] *	
• Specialization: [] *	
• Clinic Name: []	
• Clinic Address: []	
[Sign Up]	
Note: Your account will be reviewed within 24 hours	

↓		
Step 2: User Submits Form		
Frontend Action:		
POST /users/signup		
Request Body (Doctor Example):		
{		
"email": "dr.ahmed@example.com",		
"password": "SecurePass123!",		
"full_name": "Dr. Ahmed Khan",		
"phone": "+92-300-1234567",		
"role": "doctor", ← Selected role		
"registration_number": "PMC-12345", ← Doctor only		
"specialization": "Cardiologist", ← Doctor only		
"clinic_name": "City Hospital", ← Doctor only		
"clinic_address": "Main St, Karachi" ← Doctor only		
}		
Request Body (Staff Example):		
{		
"email": "john.doe@example.com",		
"password": "SecurePass123!",		
"full_name": "John Doe",		
"phone": "+92-300-7654321",		
"role": "staff" ← Selected role		
// No doctor fields		
}		
↓		
Step 3: Backend Processes Signup		
Backend (app/api/routes/users.py):		
@router.post("/users/signup")		
async def register_user(user_in: UserRegister, db: Session):		
# Validation and database logic		
# 1. Validation Checks		
# • Check if email already exists		

- Validate password strength (min 8 chars)
- Validate email format
- If doctor: validate registration_number exists

↓

2. Create User Record in Database

```
user = User(  
    email = "dr.ahmed@example.com"  
    hashed_password = hash("SecurePass123!")  
    full_name = "Dr. Ahmed Khan"  
    phone = "+92-300-1234567"  
    role = "doctor" OR "staff"  
  
    // Doctor-specific fields (only if role=doctor)  
    registration_number = "PMC-12345"  
    specialization = "Cardiologist"  
    clinic_name = "City Hospital"  
    clinic_address = "Main St, Karachi"  
  
    // Status flags - CRITICAL!  
    is_verified = FALSE ← Email not verified yet  
    is_approved = FALSE ← Admin approval pending  
    is_active = FALSE ← Cannot login yet  
  
    created_at = 2026-02-06 10:30:00  
)  
  
db.add(user)  
db.commit()
```

↓

3. Generate Verification Token

```
token = generate_verification_token(user.email)  
// Token format: JWT with 24-hour expiry  
// Contains: email, timestamp, signature
```

↓

4. Send Verification Email to User

TO: dr.ahmed@example.com

SUBJECT: "Verify Your Email - Spine Homeo"

BODY:

"Hi Dr. Ahmed Khan,

Thanks for signing up! Please verify your email:

[Verify Email] → <https://app.com/verify-email?token=xyz...>

This link expires in 24 hours."



5. Notify Admin via Email

TO: admin@clinic.com

SUBJECT: "🔔 New Doctor Signup: Dr. Ahmed Khan"

BODY:

"New doctor signup pending approval:

👤 Name: Dr. Ahmed Khan

✉ Email: dr.ahmed@example.com

📞 Phone: +92-300-1234567

🏥 Specialization: Cardiologist

🆔 Registration: PMC-12345

📍 Clinic: City Hospital

📅 Signed up: Feb 6, 2026 10:30 AM

✉ Email verified: Pending

[Review in Dashboard] → <https://admin.app.com/pending>



6. Return Success Response

HTTP 200 OK	
{	
"id": "550e8400-e29b-41d4-a716-446655440000",	
"email": "dr.ahmed@example.com",	
"full_name": "Dr. Ahmed Khan",	
"role": "doctor",	
"is_verified": false,	
"is_approved": false,	
"is_active": false	
}	
↓	
Step 4: User Sees Success Message	
Frontend shows:	
✔ Signup Successful! We've sent a verification link to: dr.ahmed@example.com Please check your email and click the verification link. [Go to Login]	
User redirected to: /login (but can't login yet)	
PHASE 2: EMAIL VERIFICATION (User Action Required)	
Step 5: User Checks Email	
User receives email:	

From: Spine Homeo <noreply@spinehomeo.com>
To: dr.ahmed@example.com
Subject: Verify Your Email - Spine Homeo

Hi Dr. Ahmed Khan,

Thanks for signing up as a Doctor on Spine Homeo!

Please verify your email address by clicking the link below:

[Verify Email Address]
<https://app.com/verify-email?token=eyJhbGciOiJIUzI1NiIs...>

This link will expire in 24 hours.

If you didn't create this account, please ignore this email.

User clicks: [Verify Email Address]



Step 6: Email Verification Processing

User is redirected to:
GET /verify-email?token=eyJhbGciOiJIUzI1NiIs...

Backend processes:

```
@router.post("/verify-email/{token}")
async def verify_email(token: str, db: Session):
    1. Decode and validate token
    2. Extract email from token
    3. Find user in database
    4. Update: is_verified = TRUE
    5. db.commit()

    return {"message": "Email verified successfully"}
```

Database updated:

User record now:

is_verified = TRUE  Email confirmed!	
is_approved = FALSE  Still pending admin approval	
is_active = FALSE  Still cannot login	
↓	
Step 7: User Sees Verification Success	
 Email Verified Successfully! Your email has been confirmed.  Your account is now under review by our admin team. You'll receive an email notification once approved (usually within 24 hours). [Go to Login]	
↓	
Step 8: User Tries to Login (WILL FAIL)	
User goes to: /login Enters credentials: Email: dr.ahmed@example.com Password: SecurePass123! Backend checks:	
@router.post("/login") async def login(credentials: LoginRequest, db: Session): user = authenticate_user(email, password) #  Valid if not user.is_verified:  "Please verify your email"	

if not user.is_approved: # ← FAILS HERE

❌ "Your account is pending admin approval"

if not user.is_active:

❌ "Account inactive"

Response:

HTTP 403 Forbidden

```
{
  "detail": "Your doctor account is pending admin approval.
            You will receive an email notification once approved
            (usually within 24 hours)."
```

User sees:

⌚ Account Pending Approval

Your doctor account is under review.

You'll receive an email when approved (usually within 24 hours).

PHASE 3: ADMIN REVIEW & APPROVAL (Admin Action Required)

Step 9: Admin Receives Notification

Admin checks email inbox:

From: Spine Homeo System <system@spinehomeo.com>

To: admin@clinic.com

Subject: 🛎️ New Doctor Signup: Dr. Ahmed Khan

Hello Admin,

A new doctor has signed up and is waiting for approval:

	Name: Dr. Ahmed Khan		
	Email: dr.ahmed@example.com		
	Phone: +92-300-1234567		
	Specialization: Cardiologist		
	Registration #: PMC-12345		
	Clinic: City Hospital, Main St, Karachi		
	Signed up: Feb 6, 2026 10:30 AM		
	Email verified: Yes 		
	To review and approve, visit your admin dashboard:		
	[Review in Dashboard] → https://admin.app.com/pending-approvals		

--	--	--	--

Step 10: Admin Opens Dashboard			
--------------------------------	--	--	--

Admin navigates to: https://admin.app.com/pending-approvals			
Frontend loads:			
GET /admin/pending-approvals?role=all			
Backend returns:			
{			
"data": [			
{			
"id": "550e8400...",			
"full_name": "Dr. Ahmed Khan",			
"email": "dr.ahmed@example.com",			
"phone": "+92-300-1234567",			
"role": "doctor",			
"specialization": "Cardiologist",			
"registration_number": "PMC-12345",			
"clinic_name": "City Hospital",			
"is_verified": true,			
"is_approved": false,			
"created_at": "2026-02-06T10:30:00"			
}			
],			
"count": 1			
}			

Step 11: Admin Views Pending Users

Admin sees dashboard:



User Approval Dashboard

Total: 5 | Doctors:3 | Staff: 2 | Today: 2

Show: [All] [Doctors] [Staff]

Role	Name	Email	Details	Actions
------	------	-------	---------	---------

[DR]	Dr. Ahmed Khan	dr.ahmed@...	Cardiologist	[✓] [X]
		PMC-12345		

[ST]	John Doe	john@...	Staff	[✓] [X]
------	----------	----------	-------	---------

Admin reviews Dr. Ahmed Khan's information:

- Medical registration number looks valid
- Email verified
- Specialization appropriate
- All fields filled properly



Step 12A: Admin APPROVES User

Admin clicks: [✓ Approve] button

Confirmation dialog:



Your account details:	
	Email: dr.ahmed@example.com
	Role: Doctor
	Specialization: Cardiologist
Welcome to Spine Homeo!	

Response to admin:	
HTTP 200 OK	
{	
"success": true,	
"message": "✔ Doctor 'Dr. Ahmed Khan' has been approved	
and can now login",	
"user": { ... updated user object ... }	
}	
Admin sees success notification:	

✔ Dr. Ahmed Khan has been approved!	
-------------------------------------	--

User removed from pending list (page refreshes)	
---	--

OR

Step 12B: Admin REJECTS User (Alternative Path)	
---	--

Admin clicks: [X Reject] button	
Rejection modal appears:	

Reject User Application	
Please provide a reason for rejection:	

Invalid medical registration number.

Please provide a valid PMC registration.

Admin enters reason and clicks: [Reject User]

Frontend sends:

POST /admin/approve/550e8400-e29b-41d4-a716-446655440000

```
{
  "action": "reject",
  "reason": "Invalid medical registration number..."
}
```

Backend processes:

1. Find user by ID
2. Update user:
 - is_approved = FALSE
 - is_active = FALSE
 - rejection_reason = "Invalid medical..."
3. db.commit()
4. Send rejection email to user
5. Log audit trail

Backend sends rejection email:

To: dr.ahmed@example.com

Subject: Account Application Update

Hi Dr. Ahmed Khan,

Thank you for your interest in joining Spine Homeo.

Unfortunately, we are unable to approve your account at this time.

Reason: Invalid medical registration number. Please provide a valid PMC registration.

If you believe this is an error or would like to provide					
additional information, please contact support@spinehomeo.com					
PHASE 4: USER LOGIN (After Approval)					
Step 13: User Receives Approval Email					
User checks email and sees approval notification					
User clicks: [Login Now] button in email					
↓					
Step 14: User Logs In Successfully					
User lands on: /login					
Enters credentials:					
Email: dr.ahmed@example.com					
Password: SecurePass123!					
Frontend sends:					
POST /login					
{					
"username": "dr.ahmed@example.com",					
"password": "SecurePass123!"					
}					
Backend processes:					
@router.post("/login")					
async def login(credentials: LoginRequest, db: Session):					
user = authenticate_user(email, password)					
✅ User found, password correct					
Checks:					
✅ is_verified = TRUE (Email verified)					
✅ is_approved = TRUE (Admin approved)					

☒ is_active = TRUE (Account active)

All checks pass! ☒

Generate JWT token:

```
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",
  "role": "doctor",
  "entity": "user",
  "exp": 1738850100 // 1 hour from now
}
```

Update last_login timestamp
db.commit()

Response:

HTTP 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": "550e8400...",
    "email": "dr.ahmed@example.com",
    "full_name": "Dr. Ahmed Khan",
    "role": "doctor",
    "specialization": "Cardiologist",
    "is_active": true,
    "is_verified": true,
    "is_approved": true
  }
}
```



Step 15: User Redirected to Dashboard

Frontend:

- Stores access token in localStorage
- Redirects to /dashboard

User sees their dashboard:

Welcome, Dr. Ahmed Khan! 🎉

Your account is now active. You can:

• Manage patient appointments

• View patient records

• Update your profile

• And more...

SUCCESS! User is now fully onboarded and can use the system

SUMMARY: Database State Changes Throughout Flow

After Signup (Step 3):

is_verified	is_approved	is_active	Can Login?
FALSE ❌	FALSE ❌	FALSE ❌	NO ❌

After Email Verification (Step 6):

is_verified	is_approved	is_active	Can Login?
TRUE ✅	FALSE ❌	FALSE ❌	NO ❌

Error: "Your account is pending admin approval"

After Admin Approval (Step 12A):

is_verified	is_approved	is_active	Can Login?
TRUE ✅	TRUE ✅	TRUE ✅	YES ✅

Success: User can now login and access system

After Admin Rejection (Step 12B):

is_verified	is_approved	is_active	Can Login?
TRUE ✅	FALSE ❌	FALSE ❌	NO ❌

Error: "Your account was rejected: [reason]"

KEY ENDPOINTS USED IN THIS FLOW

- 1. POST /users/signup
→ Create new user account (doctor or staff)
- 2. POST /verify-email/{token}
→ Verify user's email address
- 3. GET /admin/pending-approvals?role=all|doctor|staff
→ Get list of users pending approval (admin only)
- 4. POST /admin/approve/{user_id}
→ Approve or reject user (admin only)
- 5. POST /login
→ Authenticate user and return JWT token
- 6. GET /users/me
→ Get current user's profile (authenticated)

TIMELINE EXAMPLE

Day 1, 10:30 AM: User signs up
Day 1, 10:31 AM: User verifies email
Day 1, 10:32 AM: User tries to login → FAILS (pending approval)
Day 1, 2:00 PM: Admin reviews application
Day 1, 2:15 PM: Admin approves
Day 1, 2:16 PM: User receives approval email
Day 1, 2:20 PM: User logs in → SUCCESS 

Total time: ~4 hours (can be much faster if admin is actively monitoring)

EDGE CASES & ERROR HANDLING

- Case 1: User signs up with existing email
→ Error: "Email already registered" (HTTP 400)
- Case 2: User tries to verify with expired token
→ Error: "Verification link expired. Please request a new one" (HTTP 400)
- Case 3: User tries to login without verifying email
→ Error: "Please verify your email first" (HTTP 403)

Case 4: User tries to login after rejection
→ Error: "Your application was rejected: [reason]. Contact support" (HTTP 403)

Case 5: Admin tries to approve already-approved user
→ Success (idempotent): "User is already approved" (HTTP 200)

Case 6: Admin tries to approve user without verified email
→ Error: "Cannot approve user whose email is not verified" (HTTP 400)

END OF FLOW